Neomage: The Open-Source Agentic AI Engine That Flutter Was Missing

How a 191,870-line Flutter package brings the power of Claude Code's architecture to every AI provider — and why it matters for the community.

FLUTTER AI AGENTS OPEN SOURCE DART

Last week we published Neomage v1.0.0 on pub.dev. It's a multi-provider agentic AI engine for Flutter — 300 Dart files, 36 built-in tools, 284 skills, 12 personality modules, and native support for Gemini, OpenAI, Anthropic, DeepSeek, Qwen, and Ollama.

But the numbers don't tell the real story. The real story is **why** Flutter needs this and **how** it changes what you can build.

The Problem: AI in Flutter is Fragmented

If you wanted to build an AI-powered app in Flutter before Neomage, you had two options:

1. **Use a single-provider SDK** — `google_generative_ai` for Gemini, a community wrapper for OpenAI, maybe roll your own for Anthropic. Your app is locked to one provider. Switching means rewriting your API layer.
2. **Build everything from scratch** — SSE parsing, streaming, tool definitions, the agentic loop, context management, permission systems. Weeks of work before you write your first feature.

Both options share the same problem: **there was no Flutter-native equivalent of what Claude Code, LangChain, or Vercel AI SDK provide on the TypeScript side.** The infrastructure for building truly agentic applications simply didn't exist in Dart.

Neomage fills that gap.

What Neomage Actually Is

Neomage is not a chatbot UI kit. It's not a wrapper around one API. It's an **engine** — the layer between your Flutter app and any AI provider that handles the hard parts:

One Interface, Any Provider

```
// Switch providers by changing one config — same interface everywhere
final provider = OpenAiShim(ApiConfig(
  type: ApiProviderType.gemini,
  apiKey: 'your-key',
  model: 'gemini-2.5-flash',
));

// Or use Anthropic
final claude = AnthropicClient(ApiConfig.anthropic(apiKey: 'sk-...'));

// Or run locally with Ollama — no API key needed
final local = OpenAiShim(ApiConfig(
  type: ApiProviderType.ollama,
  baseUrl: 'http://localhost:11434/v1',
  model: 'llama3.1:8b',
));
```

Every provider returns the same `Stream<StreamEvent>`. Your UI code doesn't know or care which model is behind it. This is the same shim pattern that Claude Code uses internally — we studied it, understood it, and ported it to Dart.

The Agentic Loop

This is the core of what makes Neomage different from a simple API wrapper. The `QueryEngine` implements a multi-turn execution loop:

```
user message → API call → tool extraction → permission check →
tool execution → result injection → compaction check → repeat
```

When you tell Neomage "read the pubspec.yaml and update the version," it doesn't respond with instructions. It **does it** — reads the file, edits it, verifies the change, and reports back. Up to 25 autonomous turns per query, with automatic context compaction when approaching token limits.

```
final engine = QueryEngine(
  provider: provider,
  toolRegistry: toolRegistry,
  systemPrompt: systemPrompt,
  compactionService: compactionService,
);

final result = await engine.query(
  messages: [Message.user('Fix the bug in auth_service.dart')],
  onTextDelta: (text) => print(text),
  onToolUse: (name, input) => print('Using: $name'),
);
```

36 Built-in Tools

Every tool has a detailed description that tells the AI model exactly how to use it — following patterns directly inspired by Claude Code's tool system:

Tool	What it does
Bash	Execute shell commands with security validation
Read	Read files with offset/limit, images, PDFs, notebooks
Write	Create files with parent directory auto-creation
Edit	Surgical string replacements with quote normalization
Grep	Ripgrep-powered regex search across codebases
Glob	Fast file pattern matching
Agent	Spawn sub-agents for parallel task execution
WebSearch	Search the web with domain filtering
WebFetch	Fetch and process web content
SendMessage	Inter-agent communication
TodoWrite	Structured task tracking
LSP	Language Server Protocol integration

Each tool description isn't just a label — it's a multi-paragraph instruction set that teaches the model when to use it, what to avoid, and how to handle edge cases. For example, the Bash tool explicitly tells the model: *"Use Glob instead of `find`. Use Grep instead of `grep`. Use Read instead of `cat`."* This level of guidance is what makes even small local models behave like capable agents.

The Personality System: Teaching Any Model to Be an Agent

Here's something most AI frameworks miss: **a small model doesn't know it can use tools unless you tell it very clearly.**

Neomage includes 12 modular personality files written in plain Markdown — no XML, no YAML, no framework-specific DSL. Just headers, bullet points, and bold text that any model from 3B parameters up can understand:

- **IDENTITY** — Who Neomage is. Strategic mentor, not servile chatbot.
- **COGNITION** — Chain of thought protocol: UNDERSTAND → PLAN → EXECUTE → VERIFY → REPORT.
- **TOOLS** — Eight critical rules, starting with: "You DO have access to the user's files. NEVER say you can't."
- **AGENCY** — Autonomy levels: what to do freely, what to confirm, what to always ask about.
- **MEMORY** — Context hierarchy and thread continuity.
- **METACOGNITION** — Self-monitoring before every response.
- **And 6 more** — Coherence, Introspection, Consolidation, Artifacts, Manus, Capabilities.

These modules are loaded at startup and assembled dynamically with session context — the model, working directory, git branch, platform, and any user-specific instructions:

```
await NeomageSystemPrompt.load();

final systemPrompt = NeomageSystemPrompt.build(
  model: 'gemini-2.5-flash',
  workingDirectory: '/home/user/my_app',
  gitBranch: 'feature/auth',
  platform: 'Darwin 24.4.0',
  isGitRepo: true,
);
```

The result is a ~2,500-word system prompt that transforms even a basic model into an agent that reads files, runs commands, and reports results — instead of saying "I can't access your filesystem."

---

284 Skills Across 40+ Categories

Skills are loadable knowledge modules — Markdown files that inject domain expertise into the active session:

- **Architecture**: Microservices, clean architecture, event-driven design
- **Flutter**: State management, navigation, platform channels
- **Testing**: Unit, integration, E2E, TDD, mutation testing
- **Security**: OWASP, authentication, secrets management
- **AI/ML**: Prompt engineering, RAG, fine-tuning
- **Git Workflow**: Branching strategies, code review, CI/CD
- **And 34 more categories**

Users browse and load skills from the Skills panel in the UI. When loaded, the skill content is injected into the system prompt, giving the model specialized knowledge for the current task.

Built for Flutter, Not Ported to Flutter

Neomage isn't a JavaScript library wrapped in a Dart binding. Every line is native Dart, built on Flutter's platform abstractions:

- **Cross-platform IO** — Conditional exports (`dart:io` on native, stubs on web) mean the same code runs on macOS, Linux, Windows, iOS, Android, and Web.
- **Sint framework** — Reactive state management with `SintController`, `.obs` observables, and `Obx()` widgets. No Provider, no Riverpod, no boilerplate.
- **Streaming** — Native Dart `Stream<StreamEvent>` with typed events (`TextDelta`, `ContentBlockStart`, `ToolUseStart`, `ThinkingDelta`). Plug directly into `StreamBuilder` or `Obx`.
- **Hive + SharedPreferences** — API keys in encrypted Hive boxes, preferences in SharedPreferences. No external backend required.

The example app in the repository is a full AI coding assistant — streaming chat, model switching, Ollama local models, command palette, vim mode, settings, onboarding, Material 3 theming — all built with Neomage as the only AI dependency.

Standing on the Shoulders of Giants

We want to be transparent about where Neomage comes from. This project wouldn't exist without:

Anthropic — For open-sourcing Claude Code. We studied every line of their 517,000-line TypeScript codebase. The tool system architecture, the system prompt design, the agentic loop, the permission model, the compaction strategies — all of it informed Neomage's design. When we say "inspired by," we mean we read `prompts.ts`, understood why each tool description is three paragraphs long, and translated that philosophy to Dart.

Google — For the Gemini API and for Flutter itself. Neomage runs on 6 platforms because Flutter exists. The Gemini client was our first provider implementation, and it remains the default for new users.

OpenAI — For establishing the chat completions API standard. Neomage's `OpenAiShim` speaks this protocol to Ollama, DeepSeek, Qwen, OpenRouter, Together, Groq, Fireworks, LM Studio, and any other compatible endpoint. One shim, dozens of providers.

Qwen (Alibaba Cloud) — For the Qwen model family and DashScope API. Having a first-class Qwen provider means Neomage works for developers in regions where other providers are less accessible.

Ollama — For making local inference accessible. Running `ollama pull llama3.1` and having a fully functional agentic AI in your Flutter app — offline, private, no API key — is the kind of developer experience we want to enable.

What You Can Build

Neomage is a package, not an app. Here's what you can build with it:

- **AI coding assistants** — The example app is one. Fork it, customize it, ship it.

- **Document processors** — Read files, analyze content, generate reports. The tool system handles the I/O.
- **DevOps dashboards** — Run commands, parse outputs, monitor systems. Bash tool + streaming = real-time ops.
- **Educational tools** — The skills system can inject domain knowledge for tutoring, code review, or exam prep.
- **Local-first AI apps** — Ollama support means full agentic capabilities without internet or API costs.
- **Multi-agent systems** — The Agent tool spawns sub-agents. SendMessage enables inter-agent communication. TodoWrite tracks distributed work.

Getting Started

```yaml
# pubspec.yaml
dependencies:
  neomage: ^1.0.0
```

```dart
import 'package:neomage/neomage.dart';

// 1. Create a provider
final provider = OpenAiShim(ApiConfig(
  type: ApiProviderType.ollama,
  baseUrl: 'http://localhost:11434/v1',
  model: 'llama3.1:8b',
));

// 2. Register tools
final registry = ToolRegistry();
registry.register(BashTool());
registry.register(FileReadTool());
registry.register(FileEditTool());
registry.register(GrepTool());

// 3. Build the system prompt
await NeomageSystemPrompt.load();
final prompt = NeomageSystemPrompt.build(
  model: 'llama3.1:8b',
  workingDirectory: Directory.current.path,
);

// 4. Create the engine and query
final engine = QueryEngine(
  provider: provider,
  toolRegistry: registry,
  systemPrompt: prompt,
);

final result = await engine.query(
  messages: [Message.user('List all Dart files in this project')],
  onTextDelta: (t) => stdout.write(t),
  onToolUse: (name, _) => print('\n[Tool: $name]'),
);
```

That's it. Your Flutter app now has an agentic AI that can read files, run commands, search code, and report results — with any provider, on any platform.

The Road Ahead

Neomage 1.0.0 is the foundation. Here's what's coming:

- **MCP server marketplace** — Connect to Model Context Protocol servers for extended capabilities (databases, APIs, browsers).
- **Multi-agent orchestration** — Coordinator mode for complex workflows with multiple specialized agents.
- **Voice input/output** — STT/TTS integration for mobile-first experiences.
- **Plugin system** — Third-party tool packages that register with the engine automatically.
- **Context-aware suggestions** — Proactive prompts based on project state and user patterns.

Join the Community

pub.dev: neomage

GitHub: Open-Neom/neomage

Publisher: openneom.dev

Neomage is MIT-licensed and part of the Open Neom ecosystem.

If you're building AI-powered Flutter apps, you no longer have to start from zero. The engine is here.

Neomage — Your AI agent to create, explore, and execute.